Authentication — b-secure Backend

App: `apps/authentication` **Covers:** Phone OTP login, JWT tokens, social auth (Google/Apple), token security

---

Table of Contents

---

1. Authentication Flow Overview

```
Phone OTP Flow:
  [1] User enters phone number
       ↓
  [2] POST /api/auth/send-otp/
       → Rate check (max 5 OTPs per phone per 10 min)
       → Generate 6-digit OTP
       → Hash OTP, store in DB (expires in 5 min)
       → Send OTP via SMS (Twilio/MSG91)
       → Return: {"message": "OTP sent", "expires_in": 300}
       ↓
  [3] User enters OTP
       ↓
  [4] POST /api/auth/verify-otp/
       → Find latest unexpired OTP for phone
       → Verify hash match
       → Mark OTP as used
       → If new phone: create UserProfile (or GuardProfile based
```

```
on role)
       → Issue JWT access token (15 min) + refresh token (30 days)
       → Return: {"access": "...", "refresh": "...", "user":
{...}, "is_new_user": true}
       ↓
  [5] Client stores tokens in Secure Storage
       ↓
  [6] All subsequent requests: Authorization: Bearer
<access_token>
       ↓
  [7] When access token expires (401 response):
       POST /api/auth/refresh/ with refresh token
       → New access token returned
       ↓
  [8] Logout: POST /api/auth/logout/
       → Refresh token blacklisted in DB
       → Client clears local tokens
```

---

2. OTP Authentication

OTP Generation & Delivery

```python
# apps/authentication/services.py

import random
import hashlib
from datetime import timedelta
from django.utils import timezone
from django.conf import settings
from .models import OTPToken


class OTPService:

    OTP_EXPIRY_SECONDS = 300       # 5 minutes
    OTP_LENGTH = 6
    MAX_ATTEMPTS = 3               # Failed attempts before
        lockout
    MAX_SENDS_PER_WINDOW = 5       # Max OTPs in 10-minute window
    SEND_WINDOW_SECONDS = 600      # 10 minutes

    @classmethod
    def send_otp(cls, phone_number: str, purpose: str = 'LOGIN')
        -> dict:
        """
        Generate and send OTP to the given phone number.
        Returns expiry info. Raises on rate limit violation.
        """
        cls._check_send_rate_limit(phone_number)
```

```
        # Expire any existing unused OTPs for this phone
        OTPToken.objects.filter(
            phone_number=phone_number,
            is_used=False,
            expires_at__gt=timezone.now()
        ).update(expires_at=timezone.now())

        # Generate new OTP
        otp_code = cls._generate_otp()
        otp_hash = cls._hash_otp(otp_code)
        expires_at = timezone.now() +
        timedelta(seconds=cls.OTP_EXPIRY_SECONDS)

        OTPToken.objects.create(
            phone_number=phone_number,
            otp_hash=otp_hash,
            expires_at=expires_at,
            purpose=purpose,
        )

        # Send via SMS (async Celery task)
        from apps.notifications.tasks import send_otp_sms
        send_otp_sms.apply_async(
            args=[phone_number, otp_code],
            queue='high_priority'
        )

        return {
            'message': 'OTP sent successfully',
            'expires_in': cls.OTP_EXPIRY_SECONDS,
        }

    @classmethod
    def verify_otp(cls, phone_number: str, otp_code: str,
        purpose: str = 'LOGIN') -> bool:
        """
        Verify an OTP code for the given phone number.
        Marks token as used on success.
        Raises ValueError on failure with specific reason.
        """
        token = OTPToken.objects.filter(
            phone_number=phone_number,
            is_used=False,
            purpose=purpose,
        ).order_by('-created_at').first()

        if not token:
            raise ValueError('NO_OTP_FOUND')

        if token.expires_at < timezone.now():
            raise ValueError('OTP_EXPIRED')

        if token.attempt_count >= cls.MAX_ATTEMPTS:
```

```
            raise ValueError('MAX_ATTEMPTS_EXCEEDED')

        if token.otp_hash != cls._hash_otp(otp_code):
            token.attempt_count += 1
            token.save(update_fields=['attempt_count'])
            remaining = cls.MAX_ATTEMPTS - token.attempt_count
            raise ValueError(f'INVALID_OTP:{remaining}')

        # Success — mark as used
        token.is_used = True
        token.save(update_fields=['is_used'])
        return True

    @classmethod
    def _generate_otp(cls) -> str:
        return str(random.SystemRandom().randint(
            10 ** (cls.OTP_LENGTH - 1),
            10 ** cls.OTP_LENGTH - 1
        ))

    @classmethod
    def _hash_otp(cls, otp: str) -> str:
        return hashlib.sha256(otp.encode('utf-8')).hexdigest()

    @classmethod
    def _check_send_rate_limit(cls, phone_number: str):
        window_start = timezone.now() -
        timedelta(seconds=cls.SEND_WINDOW_SECONDS)
        recent_count = OTPToken.objects.filter(
            phone_number=phone_number,
            created_at__gte=window_start,
        ).count()
        if recent_count >= cls.MAX_SENDS_PER_WINDOW:
            raise PermissionError('RATE_LIMIT_EXCEEDED')
```

OTP Verification & Token Issuance

```
# apps/authentication/services.py (continued)

from rest_framework_simplejwt.tokens import RefreshToken
from apps.users.models import UserProfile
from apps.guards.models import GuardProfile


class AuthService:

    @classmethod
    def authenticate_user(cls, phone_number: str, role: str =
        'USER') -> dict:
        """
        Called after OTP verification succeeds.
```

```
        Creates user/guard if first time, returns JWT tokens +
        profile.
        """
        is_new = False

        if role == 'GUARD':
            user, is_new =
        cls._get_or_create_guard_user(phone_number)
        else:
            user, is_new = cls._get_or_create_user(phone_number)

        # Issue JWT tokens
        refresh = RefreshToken.for_user(user)
        access = refresh.access_token

        # Add custom claims to access token
        access['role'] = role
        access['user_id'] = str(user.id)

        return {
            'access': str(access),
            'refresh': str(refresh),
            'is_new_user': is_new,
            'role': role,
            'user_id': str(user.id),
        }

    @classmethod
    def _get_or_create_user(cls, phone_number: str):
        return UserProfile.objects.get_or_create(
            phone_number=phone_number,
            defaults={
                'username': phone_number,
                'role': 'USER',
            }
        )

    @classmethod
    def _get_or_create_guard_user(cls, phone_number: str):
        user, created = UserProfile.objects.get_or_create(
            phone_number=phone_number,
            defaults={
                'username': phone_number,
                'role': 'GUARD',
            }
        )
        if created:
            GuardProfile.objects.create(user=user)
        return user, created
```

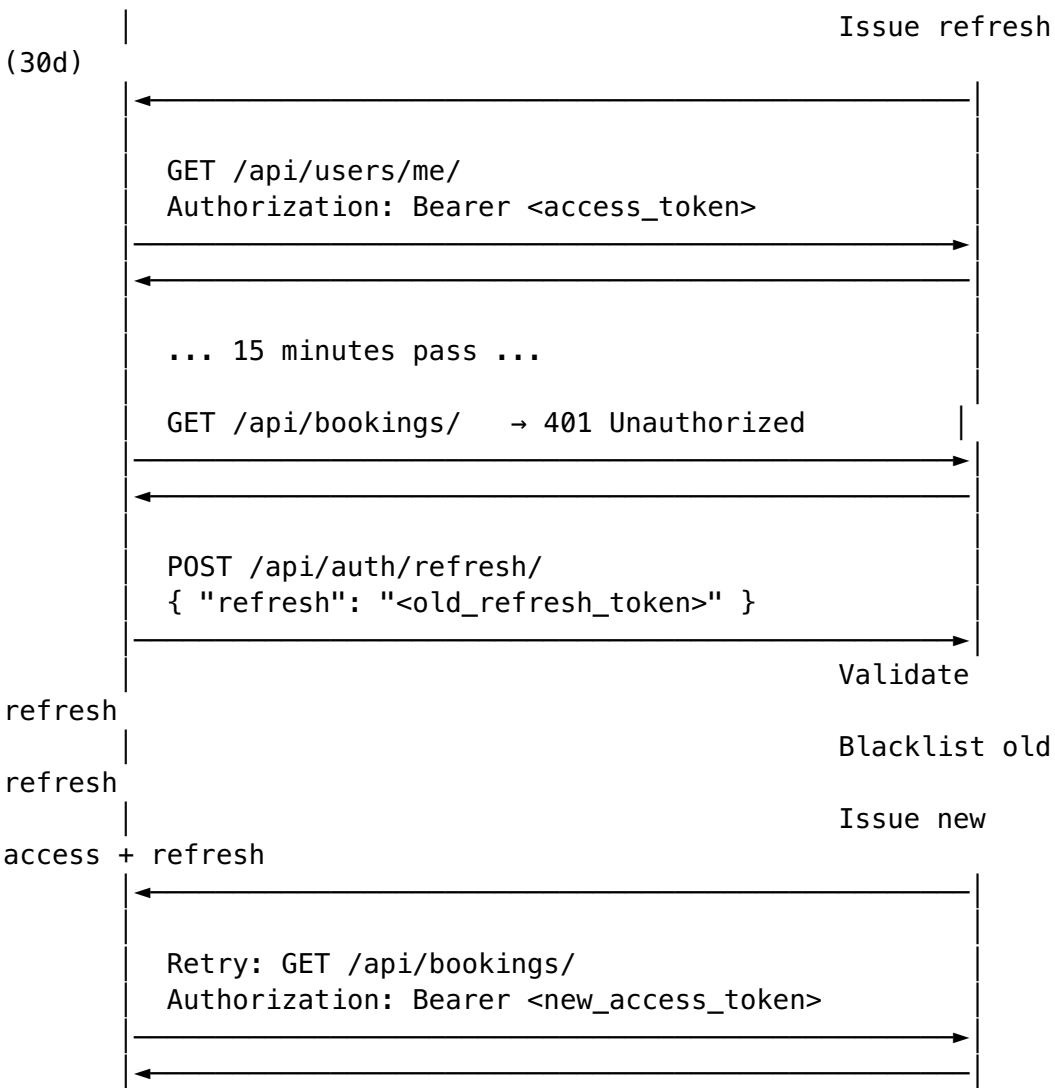
---

3. JWT Token Strategy

Configuration

```python
# config/settings/base.py
from datetime import timedelta

SIMPLE_JWT = {
    # Access token: short-lived, used on every API request
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15),

    # Refresh token: long-lived, used only to get new access
        tokens
    'REFRESH_TOKEN_LIFETIME': timedelta(days=30),

    # Rotate refresh token on every use (rolling window)
    'ROTATE_REFRESH_TOKENS': True,

    # Blacklist old refresh tokens after rotation
    'BLACKLIST_AFTER_ROTATION': True,

    # Update last_login on token refresh
    'UPDATE_LAST_LOGIN': True,

    'ALGORITHM': 'HS256',
    'SIGNING_KEY': env('SECRET_KEY'),

    'AUTH_HEADER_TYPES': ('Bearer',),
    'AUTH_HEADER_NAME': 'HTTP_AUTHORIZATION',

    'USER_ID_FIELD': 'id',
    'USER_ID_CLAIM': 'user_id',

    # Custom token serializer to add claims
    'TOKEN_OBTAIN_SERIALIZER':
        'apps.authentication.serializers.CustomTokenObtainSerializer',
}
```

Token Lifecycle



Token Blacklisting

Uses `rest_framework_simplejwt.token_blacklist` app (included in `INSTALLED_APPS`).

- On logout: refresh token added to `OutstandingToken` / `BlacklistedToken` tables
- On token rotation: old refresh token auto-blacklisted
- Celery task runs nightly to clean up expired tokens from blacklist table

```python
# apps/authentication/views.py

from rest_framework_simplejwt.tokens import RefreshToken
from rest_framework_simplejwt.exceptions import TokenError

class LogoutView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        try:
            refresh_token = request.data.get('refresh')
```

```
            token = RefreshToken(refresh_token)
            token.blacklist()
            return Response({'message': 'Logged out
        successfully'})
        except TokenError:
            return Response(
                {'error': {'code': 'INVALID_TOKEN', 'message':
        'Invalid or expired token'}},
                status=400
            )
```

---

4. Social Authentication

Google Sign-In

```
Flow:
  [1] Mobile app gets Google ID token from Google Sign-In SDK
  [2] POST /api/auth/social/google/ { "id_token": "..." }
  [3] Backend verifies token with Google's public keys
  [4] Extract email, name, Google user ID from verified token
  [5] Get or create UserProfile
  [6] Issue JWT tokens (same as OTP flow)
```

```
# apps/authentication/services.py

from google.oauth2 import id_token as google_id_token
from google.auth.transport import requests as google_requests
from django.conf import settings


class GoogleAuthService:

    @classmethod
    def verify_and_authenticate(cls, id_token_str: str, role: str
        = 'USER') -> dict:
        try:
            id_info = google_id_token.verify_oauth2_token(
                id_token_str,
                google_requests.Request(),
                settings.GOOGLE_CLIENT_ID,
            )
        except ValueError as e:
            raise ValueError(f'INVALID_GOOGLE_TOKEN: {e}')

        google_user_id = id_info['sub']
        email = id_info.get('email', '')
        name = id_info.get('name', '')

        user, is_new = UserProfile.objects.get_or_create(
            google_id=google_user_id,
```

```
            defaults={
                'email': email,
                'full_name': name,
                'username': email or google_user_id,
                'role': role,
            }
        )

        return AuthService.issue_tokens(user, is_new, role)
```

Apple Sign-In

```
# apps/authentication/services.py

import jwt
import requests


class AppleAuthService:

    APPLE_PUBLIC_KEYS_URL = 'https://appleid.apple.com/auth/keys'
    APPLE_ISSUER = 'https://appleid.apple.com'

    @classmethod
    def verify_and_authenticate(cls, identity_token: str, role:
        str = 'USER') -> dict:
        # Fetch Apple's public keys
        keys_response = requests.get(cls.APPLE_PUBLIC_KEYS_URL)
        keys = keys_response.json()['keys']

        # Decode header to find which key to use
        header = jwt.get_unverified_header(identity_token)
        apple_key = next(k for k in keys if k['kid'] ==
        header['kid'])

        public_key =
        jwt.algorithms.RSAAlgorithm.from_jwk(apple_key)

        try:
            claims = jwt.decode(
                identity_token,
                public_key,
                algorithms=['RS256'],
                audience=settings.APPLE_CLIENT_ID,
                issuer=cls.APPLE_ISSUER,
            )
        except jwt.PyJWTError as e:
            raise ValueError(f'INVALID_APPLE_TOKEN: {e}')

        apple_user_id = claims['sub']
        email = claims.get('email', '')
```

```
        user, is_new = UserProfile.objects.get_or_create(
            apple_id=apple_user_id,
            defaults={
                'email': email,
                'username': email or apple_user_id,
                'role': role,
            }
        )

        return AuthService.issue_tokens(user, is_new, role)
```

---

5. Models

```
# apps/authentication/models.py

import uuid
from django.db import models
from django.utils import timezone


class OTPToken(models.Model):
    """
    Stores OTP tokens for phone-based authentication.
    Each OTP is hashed before storage — never stored in plaintext.
    """

    PURPOSE_CHOICES = [
        ('LOGIN', 'Login / Registration'),
        ('PHONE_CHANGE', 'Phone Number Change'),
        ('TRANSACTION', 'Payment Authorization'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
        editable=False)
    phone_number = models.CharField(max_length=20, db_index=True)
    otp_hash = models.CharField(max_length=64)
        # SHA-256 hex digest
    purpose = models.CharField(max_length=20,
        choices=PURPOSE_CHOICES, default='LOGIN')
    is_used = models.BooleanField(default=False)
    attempt_count = models.PositiveSmallIntegerField(default=0)
    expires_at = models.DateTimeField()
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'auth_otp_token'
        indexes = [
            models.Index(fields=['phone_number', 'is_used',
        'expires_at']),
        ]
        ordering = ['-created_at']
```

```
    def is_valid(self) -> bool:
        return not self.is_used and self.expires_at >
        timezone.now()

    def __str__(self):
        return f'OTP for {self.phone_number} [{self.purpose}] -
        used={self.is_used}'
```

---

6. Serializers

```
# apps/authentication/serializers.py

from rest_framework import serializers
import re


PHONE_REGEX = re.compile(r'^\+[1-9]\d{6,14}$')  # E.164 format


class SendOTPSerializer(serializers.Serializer):
    phone_number = serializers.CharField(max_length=20)
    role = serializers.ChoiceField(choices=['USER', 'GUARD'],
        default='USER')

    def validate_phone_number(self, value):
        # Normalize: strip spaces, ensure E.164 format
        value = value.strip().replace(' ', '').replace('-', '')
        if not PHONE_REGEX.match(value):
            raise serializers.ValidationError(
                'Enter a valid phone number in E.164 format (e.g.
        +919876543210)'
            )
        return value


class VerifyOTPSerializer(serializers.Serializer):
    phone_number = serializers.CharField(max_length=20)
    otp_code = serializers.CharField(min_length=4, max_length=8)
    role = serializers.ChoiceField(choices=['USER', 'GUARD'],
        default='USER')

    def validate_phone_number(self, value):
        value = value.strip().replace(' ', '').replace('-', '')
        if not PHONE_REGEX.match(value):
            raise serializers.ValidationError('Invalid phone
        number format.')
        return value

    def validate_otp_code(self, value):
        if not value.isdigit():
```

```
            raise serializers.ValidationError('OTP must contain
        digits only.')
        return value


class TokenRefreshSerializer(serializers.Serializer):
    refresh = serializers.CharField()


class GoogleAuthSerializer(serializers.Serializer):
    id_token = serializers.CharField()
    role = serializers.ChoiceField(choices=['USER', 'GUARD'],
        default='USER')


class AppleAuthSerializer(serializers.Serializer):
    identity_token = serializers.CharField()
    role = serializers.ChoiceField(choices=['USER', 'GUARD'],
        default='USER')
    # Apple only sends name/email on first sign-in
    full_name = serializers.CharField(required=False,
        allow_blank=True)
```

---

7. Views & URL Endpoints

```
# apps/authentication/views.py

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import AllowAny, IsAuthenticated
from rest_framework import status
from rest_framework_simplejwt.tokens import RefreshToken
from rest_framework_simplejwt.exceptions import TokenError

from .serializers import (
    SendOTPSerializer, VerifyOTPSerializer,
    GoogleAuthSerializer, AppleAuthSerializer,
)
from .services import OTPService, AuthService, GoogleAuthService,
        AppleAuthService
from .throttles import OTPSendThrottle


class SendOTPView(APIView):
    """
    POST /api/auth/send-otp/
    Generates and sends a 6-digit OTP to the provided phone
        number.
    Rate limited to 5 requests per 10 minutes per phone number.
    """
    permission_classes = [AllowAny]
```

```
    throttle_classes = [OTPSendThrottle]

    def post(self, request):
        serializer = SendOTPSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        phone = serializer.validated_data['phone_number']
        role = serializer.validated_data['role']

        try:
            result = OTPService.send_otp(phone, purpose='LOGIN')
            return Response({'data': result},
        status=status.HTTP_200_OK)
        except PermissionError:
            return Response(
                {'error': {'code': 'RATE_LIMIT_EXCEEDED',
                           'message': 'Too many OTP requests. Try
        again in 10 minutes.'}},
                status=status.HTTP_429_TOO_MANY_REQUESTS
            )


class VerifyOTPView(APIView):
    """
    POST /api/auth/verify-otp/
    Verifies OTP and returns JWT access + refresh tokens.
    Creates user account on first login.
    """
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = VerifyOTPSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        phone = serializer.validated_data['phone_number']
        otp_code = serializer.validated_data['otp_code']
        role = serializer.validated_data['role']

        try:
            OTPService.verify_otp(phone, otp_code)
        except ValueError as e:
            error_code = str(e).split(':')[0]
            messages = {
                'NO_OTP_FOUND': 'No OTP found. Please request a
        new one.',
                'OTP_EXPIRED': 'OTP has expired. Please request a
        new one.',
                'MAX_ATTEMPTS_EXCEEDED': 'Too many failed
        attempts. Request a new OTP.',
                'INVALID_OTP:2': 'Incorrect OTP. 2 attempts
        remaining.',
                'INVALID_OTP:1': 'Incorrect OTP. 1 attempt
        remaining.',
```

```
                'INVALID_OTP:0': 'Incorrect OTP. No attempts
        remaining.',
            }
            return Response(
                {'error': {
                    'code': error_code,
                    'message': messages.get(str(e), 'Invalid
        OTP.')
                }},
                status=status.HTTP_400_BAD_REQUEST
            )

        tokens = AuthService.authenticate_user(phone, role)
        return Response({'data': tokens},
        status=status.HTTP_200_OK)


class TokenRefreshView(APIView):
    """
    POST /api/auth/refresh/
    Returns a new access token using a valid refresh token.
    Old refresh token is blacklisted (rotation).
    """
    permission_classes = [AllowAny]

    def post(self, request):
        refresh_token_str = request.data.get('refresh')
        if not refresh_token_str:
            return Response(
                {'error': {'code': 'MISSING_TOKEN', 'message':
        'Refresh token is required.'}},
                status=status.HTTP_400_BAD_REQUEST
            )
        try:
            refresh = RefreshToken(refresh_token_str)
            data = {
                'access': str(refresh.access_token),
                'refresh': str(refresh),  # New rotated refresh
        token
            }
            return Response({'data': data})
        except TokenError as e:
            return Response(
                {'error': {'code': 'INVALID_TOKEN', 'message':
        str(e)}},
                status=status.HTTP_401_UNAUTHORIZED
            )


class LogoutView(APIView):
    """
    POST /api/auth/logout/
    Blacklists the refresh token, effectively ending the session.
    """
```

```
    permission_classes = [IsAuthenticated]

    def post(self, request):
        try:
            refresh_token = request.data.get('refresh')
            token = RefreshToken(refresh_token)
            token.blacklist()
            return Response({'data': {'message': 'Logged out
        successfully.'}})
        except TokenError:
            return Response(
                {'error': {'code': 'INVALID_TOKEN', 'message':
        'Invalid or already used token.'}},
                status=status.HTTP_400_BAD_REQUEST
            )


class GoogleAuthView(APIView):
    """POST /api/auth/social/google/"""
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = GoogleAuthSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        try:
            tokens = GoogleAuthService.verify_and_authenticate(

        id_token_str=serializer.validated_data['id_token'],
                role=serializer.validated_data['role'],
            )
            return Response({'data': tokens})
        except ValueError as e:
            return Response(
                {'error': {'code': 'INVALID_TOKEN', 'message':
        str(e)}},
                status=400
            )


class AppleAuthView(APIView):
    """POST /api/auth/social/apple/"""
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = AppleAuthSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        try:
            tokens = AppleAuthService.verify_and_authenticate(

        identity_token=serializer.validated_data['identity_token'],
                role=serializer.validated_data['role'],
            )
            return Response({'data': tokens})
```

```
        except ValueError as e:
            return Response(
                {'error': {'code': 'INVALID_TOKEN', 'message':
        str(e)}},
                status=400
            )

# apps/authentication/urls.py

from django.urls import path
from .views import (
    SendOTPView, VerifyOTPView, TokenRefreshView,
    LogoutView, GoogleAuthView, AppleAuthView,
)

urlpatterns = [
    path('send-otp/', SendOTPView.as_view(), name='auth-send-
        otp'),
    path('verify-otp/', VerifyOTPView.as_view(), name='auth-
        verify-otp'),
    path('refresh/', TokenRefreshView.as_view(), name='auth-token-
        refresh'),
    path('logout/', LogoutView.as_view(), name='auth-logout'),
    path('social/google/', GoogleAuthView.as_view(), name='auth-
        google'),
    path('social/apple/', AppleAuthView.as_view(), name='auth-
        apple'),
]
```

---

8. Custom Throttles

```
# apps/authentication/throttles.py

from rest_framework.throttling import SimpleRateThrottle


class OTPSendThrottle(SimpleRateThrottle):
    """
    Rate limit OTP send requests per phone number.
    Limit: 5 requests per 10 minutes.
    Cache key is based on phone number (not IP — prevents shared-
        IP issues).
    """
    scope = 'otp_send'
    rate = '5/10min'

    def get_cache_key(self, request, view):
        phone = request.data.get('phone_number', '')
        if not phone:
            return self.get_ident(request)  # Fall back to IP
        return f'throttle_otp_{phone}'
```

```
class OTPVerifyThrottle(SimpleRateThrottle):
    """
    Rate limit OTP verification attempts per phone number.
    Limit: 10 attempts per 10 minutes.
    """
    scope = 'otp_verify'
    rate = '10/10min'

    def get_cache_key(self, request, view):
        phone = request.data.get('phone_number', '')
        if not phone:
            return self.get_ident(request)
        return f'throttle_otp_verify_{phone}'

# In config/settings/base.py REST_FRAMEWORK config, add:
'DEFAULT_THROTTLE_RATES': {
    'anon': '60/minute',
    'user': '300/minute',
    'otp_send': '5/10min',
    'otp_verify': '10/10min',
},
```

---

9. WebSocket Authentication

WebSocket connections cannot use standard `Authorization` headers. b-secure uses JWT in the WebSocket URL query parameter, validated in a custom ASGI middleware.

```
# utils/ws_middleware.py

from urllib.parse import parse_qs
from channels.middleware import BaseMiddleware
from channels.db import database_sync_to_async
from django.contrib.auth.models import AnonymousUser
from rest_framework_simplejwt.tokens import AccessToken
from rest_framework_simplejwt.exceptions import TokenError
from apps.users.models import UserProfile


@database_sync_to_async
def get_user_from_token(token_str: str):
    try:
        token = AccessToken(token_str)
        user_id = token['user_id']
        return
        UserProfile.objects.select_related('guard_profile').get(id=user_id)
    except (TokenError, UserProfile.DoesNotExist):
        return AnonymousUser()
```

```python
class JWTAuthMiddleware(BaseMiddleware):
    """
    Extracts JWT from WebSocket URL query param ?
        token=<access_token>
    Sets scope['user'] for use in consumers.
    """
    async def __call__(self, scope, receive, send):
        query_string = scope.get('query_string', b'').decode()
        params = parse_qs(query_string)
        token_list = params.get('token', [None])
        token_str = token_list[0] if token_list else None

        if token_str:
            scope['user'] = await get_user_from_token(token_str)
        else:
            scope['user'] = AnonymousUser()

        return await super().__call__(scope, receive, send)


def JWTAuthMiddlewareStack(inner):
    return JWTAuthMiddleware(inner)
```

Usage in WebSocket consumers:

```python
# apps/tracking/consumers.py (partial)

class TrackingConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        user = self.scope['user']

        if user.is_anonymous:
            await self.close(code=4001)  # Unauthorized
            return

        # ... rest of connection logic
```

Client-side WebSocket URL:

```
wss://api.bsecure.in/ws/tracking/booking-uuid-here/?
token=eyJhbGci...
```

---

10. Security Considerations

Threat	Mitigation
OTP brute force	Max 3 attempts per OTP token, max 5 OTP sends per 10 min per phone

Threat	Mitigation
OTP interception	OTP sent via encrypted SMS channel; stored as SHA-256 hash (not plaintext)
JWT theft	Short access token lifetime (15 min); refresh token rotation; HTTPS only
Replay attacks	Refresh tokens blacklisted after single use (rotation); OTP marked used immediately
Social token spoofing	Google/Apple tokens verified against issuer's public keys server-side
Account enumeration	Same response for "phone not found" and "OTP sent" (don't reveal existence)
Mass OTP send (abuse)	Phone-based rate limiting, not just IP-based
Credential stuffing	No password to stuff; OTP + phone required
Token leakage in logs	Never log JWT tokens, OTP codes, or phone numbers in production logs
WebSocket auth bypass	JWT validated in ASGI middleware before consumer even instantiates

Production Checklist

- [ ] `SECRET_KEY` is at least 50 characters, randomly generated, stored in Secrets Manager
- [ ] OTP codes never logged (check all Celery task logs)
- [ ] `DEBUG=False` in production
- [ ] JWT `SIGNING_KEY` rotated if compromised (requires forced re-login for all users)
- [ ] SMS provider has fraud detection enabled (Twilio Fraud Guard)
- [ ] HTTPS enforced at Nginx level, HSTS header set

- [ ] Admin endpoints IP-whitelisted
- [ ] Rate limiting configured in both DRF and Nginx (defense in depth)